



山东大学

信息科学与工程学院

2024 – 2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 1.1 汇编程序 Hello World

专 业 班 级 通信工程 三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 2 月 17 日

实验报告

【实验目的】

1. 掌握 window 的基本 Masm for Windows 集成实验环境2023 的使用
2. 掌握 win xp 下 DOS 环境下 masm5 的编译和
- 3.运行掌握 win7 win10 使用 DOSBox 实现虚拟 DOS 环境下 masm5 的编译和运行

【实验具体内容】

【实验 1】使用 DOSBox 实现虚拟 DOS 环境下编写

(1)

实验源代码

STACKS SEGMENT STACK ; 定义栈段, STACK 关键字告知链接器这是栈段, 同时其自动配置栈段相关寄存器。

DW 256 DUP(?) ;DW 为定义字大小, 一个字为两个字节,

DUP 是用于定义多个相同类型的变量

STACKS ENDS ;栈段定义结束

DATAS SEGMENT ; 定义数据段

STRING DB 13,10,'Hello World! ',13,10,'\$'; 定义字符串,

DB 意为多个字节数据, 13, 10 是 ASCII 中的回车符和换行符

DATAS ENDS ; 数据段结束

CODES SEGMENT ; 定义代码段

ASSUME CS:CODES,DS:DATAS ; ASSUME (伪指令) 告诉编译器某个段与寄存器的特定关系, 在这里, 告诉编译器 CS 指向 CODES, DS 指向 DATAS, 位置位于 Start 前即可

START: ; 代码段开始

MOV AX,DATAS ; 将 DATAS 指向的地址存入 AX 寄存器里, 接下

来将段地址存入 DS 寄存器（这个操作只能通过寄存器中介）以初始化数据段寄存器

```
MOV DS,AX
```

LEA DX, STRING ; 加载有效地址，用于将变量的偏移地址加载到寄存器中。这里 string（标识符），将字符串的地址加入寄存器 DX 中，准备显示。

```
MOV AH,9 ; 这里 9 是 9 号命令（显示字符串），将 9 移入 AH 中，显示字符串
```

```
INT 21H ; 调用 21H 中断来显示字符串
```

```
MOV AH,4CH ; 载入 4CH 指令以终结程序
```

```
INT 21H ; 中断以退出程序
```

```
CODES ENDS ; 代码段结束
```

```
END START ; 执行结束
```

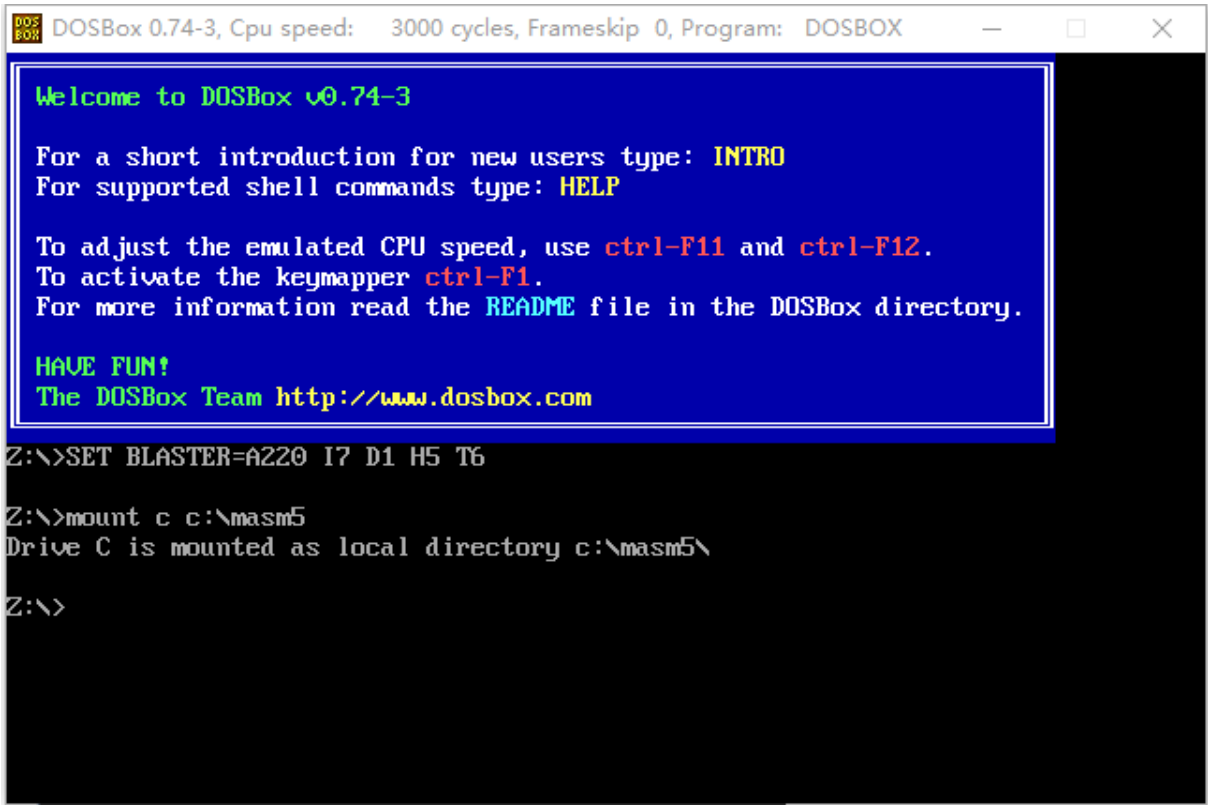
（3）实验代码、过程、相应结果（截图）：

[1] 将源代码保存至记事本，并将其后缀改为.asm 文件，并将其移入 masm5 文件夹

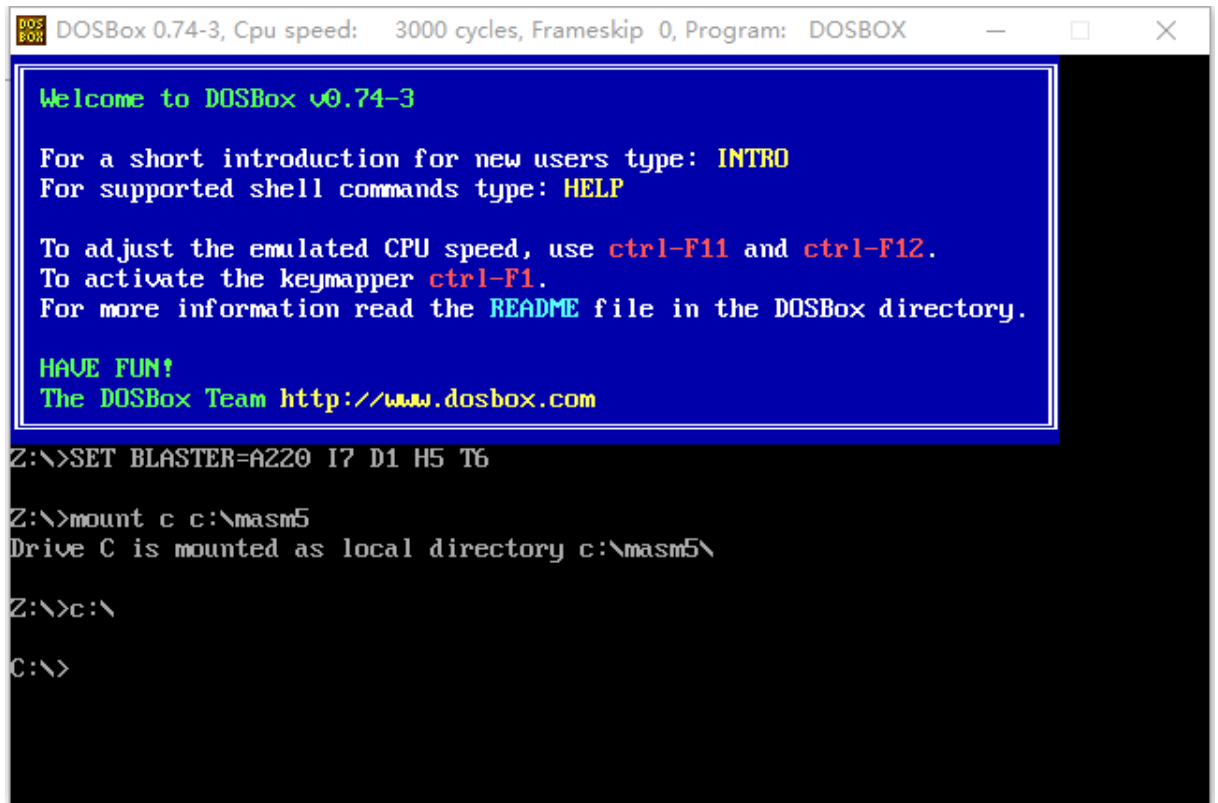


图表 1

[2] 虚拟加载 C:\（输入 **mount c:\masm5**），并进入 C:\

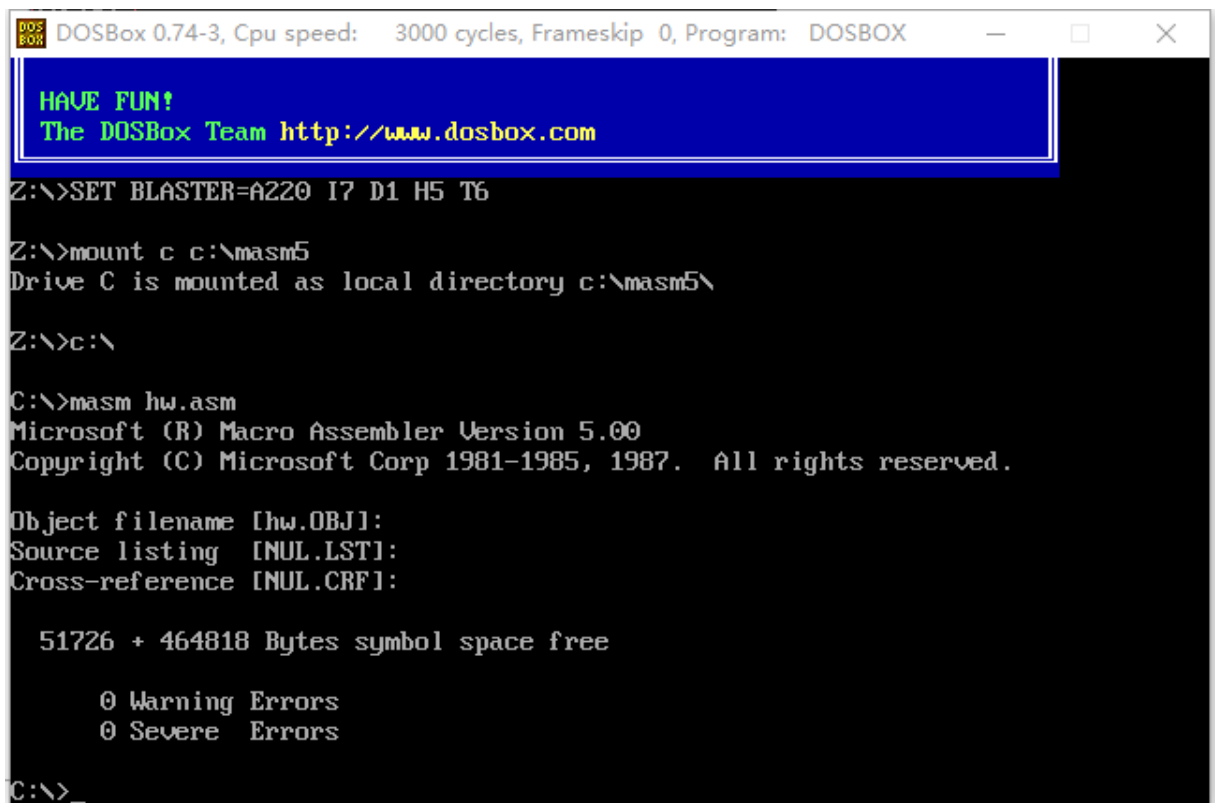


图表 2



图表 3

[3] 编译 HW.asm 文件(masm 命令), 生成.obj 对象文件



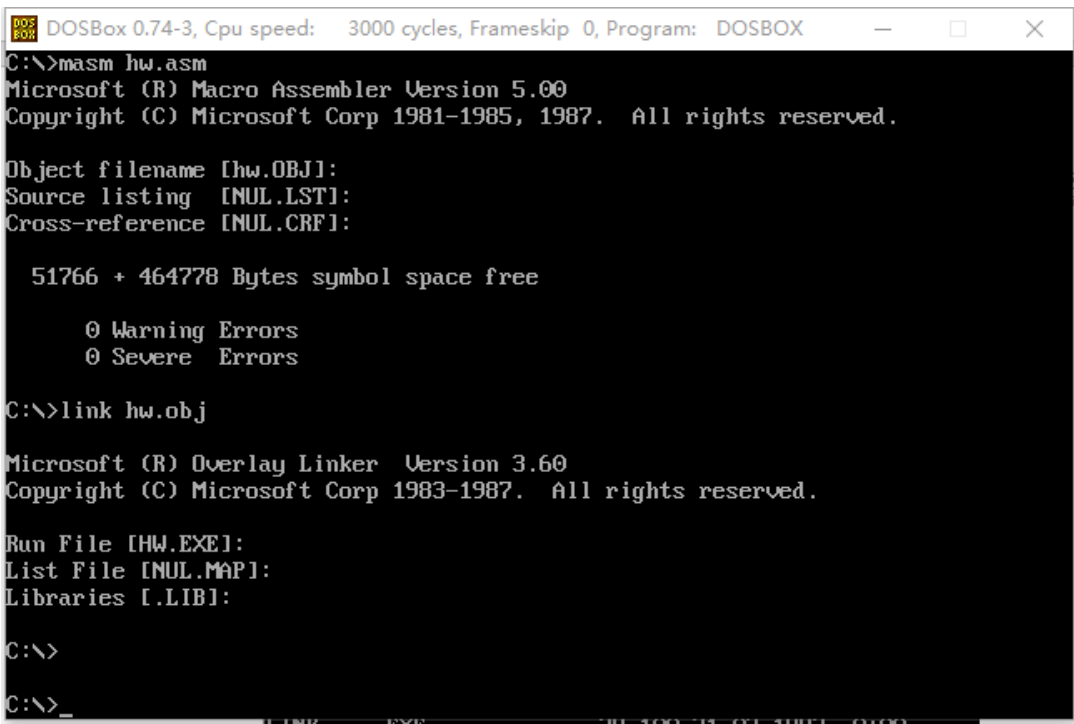
图表 4

可以发现，已生成 obj 对象文件

HW.OBJ	2025/2/17 19:23	3D Object	1 KB
LIB.EXE	1987/7/31 0:00	应用程序	32 KB
LINK.EXE	1987/7/31 0:00	应用程序	39 KB
MAKE.EXE	1987/7/31 0:00	应用程序	24 KB
MAKE	2002/10/12 11:27	指向 MS-DOS 程...	1 KB
MASM.EXE	1987/7/31 0:00	应用程序	65 KB
SETENV.EXE	1987/7/31 0:00	应用程序	11 KB

图表 5

[4] 使用 link 命令生成.exe 可执行文件



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
C:\>masm hw.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [hw.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:

51766 + 464778 Bytes symbol space free

0 Warning Errors
0 Severe Errors

C:\>link hw.obj

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [HW.EXE]:
List File [NUL.MAP]:
Libraries [LIB]:

C:\>
C:\>_
```

图表 6

若只是使用 link 命令，则会出现以下问题（如图 7）：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

0 Warning Errors
0 Severe Errors

C:\>link

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Object Modules [OBJ]:
Object Modules [OBJ]:
LINK : fatal error L1020: no object modules specified

C:\>

C:\>link

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Object Modules [OBJ]:
Object Modules [OBJ]:
LINK : fatal error L1020: no object modules specified

C:\>_

```

图表 7

发生报错，未能找到对象文件（说明我们需要指定对象文件的路径）

如果我们是直接使用 `masm5` 文件中 `HW` 来编译运行，我们会发现如下警告：（如图 8）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

EXEMOD  EXE      12,149 31-07-1987 0:00
EXEPACK  EXE      14,803 15-10-1987 5:00
HW       ASM       394 17-02-2025 19:11
HW       EXE       1,081 17-02-2025 19:26
HW       OBJ       161 17-02-2025 19:40
LIB      EXE      32,150 31-07-1987 0:00
LINK     EXE      39,100 31-07-1987 0:00
MAKE     EXE      24,199 31-07-1987 0:00
MAKE     PIF       967 12-10-2002 11:27
MASM     EXE      65,557 31-07-1987 0:00
SETENU   EXE      10,601 31-07-1987 0:00
  15 File(s)      337,845 Bytes.
   2 Dir(s)       262,111,744 Bytes free.

C:\>link hw.obj

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

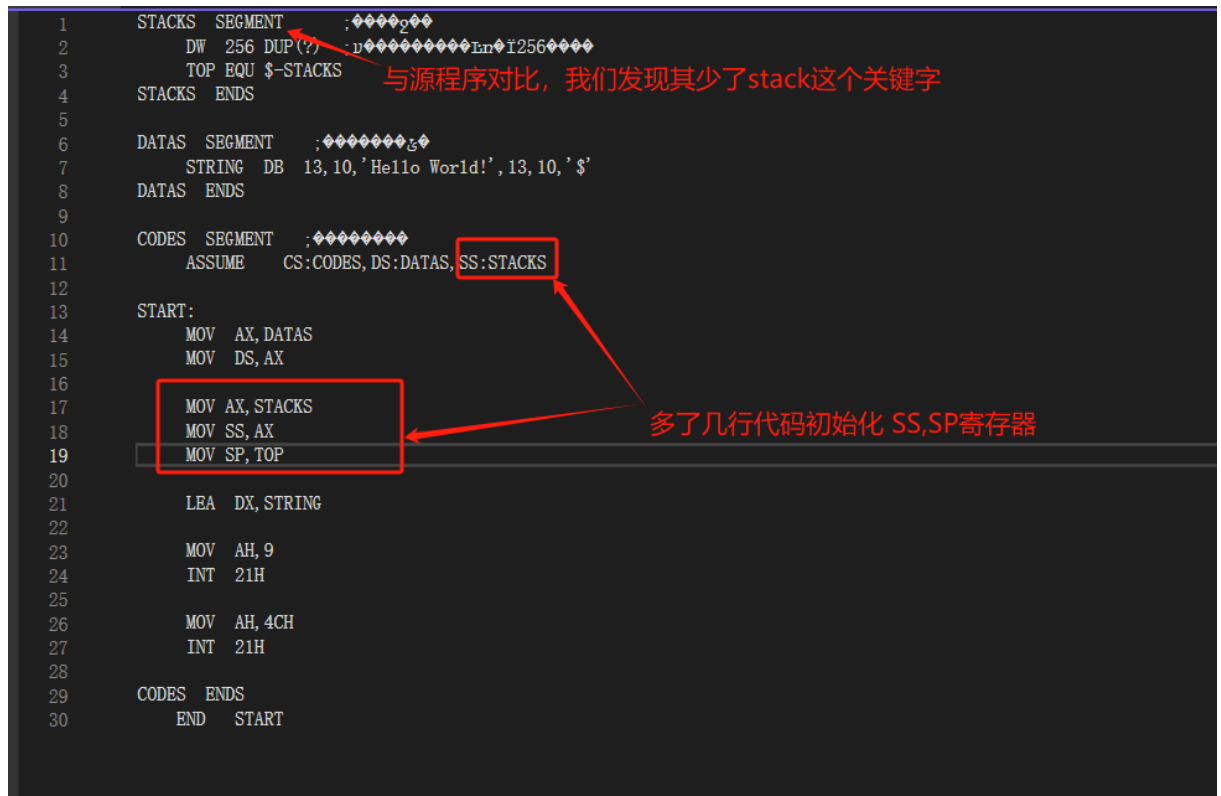
Run File [HW.EXE]:
List File [NUL.MAP]:
Libraries [LIB]:
LINK : warning L4021: no stack segment

C:\>

```

图表 8

如果我们打开对应文件，我们可以发现：



图表 9

因此, 说明栈段的定义与以下的 **DATA** 和 **CODE** 段不一样, 需要声明, 我们上网进行查询:

在 16 位汇编程序中, 链接器需要明确知道哪个段是栈段 (SS 和 SP 的初始化依赖于这一点)。如在汇编阶段未正确传递堆栈段的信息, 链接器可能会忽略它。因此我们需要显式声明 **stack** 以自动配置栈, 否则, 我们需要手动初始化寄存器以配置寄存器。

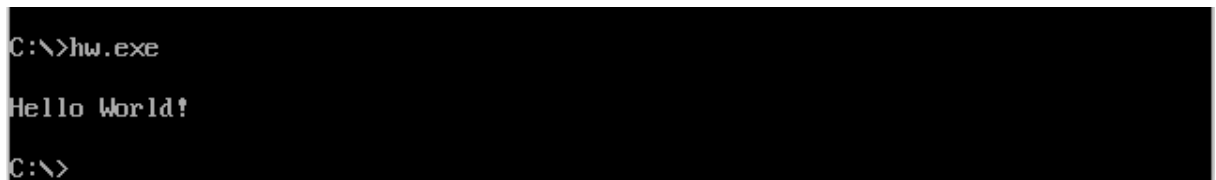
综上, 以上警告是由于连接器没有识别到栈段, 但程序中已经手动配置好了, 所以没有大的问题。

[5] 使用 **dir** 命令查看文件

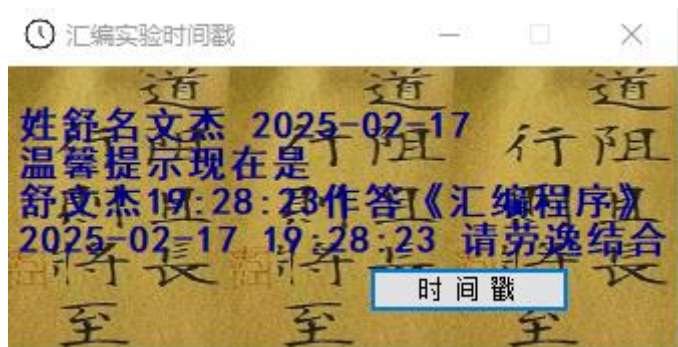


图表 10

[6] 运行 HW.exe



图表 11



图表 12

实验成功

【第二个实验】

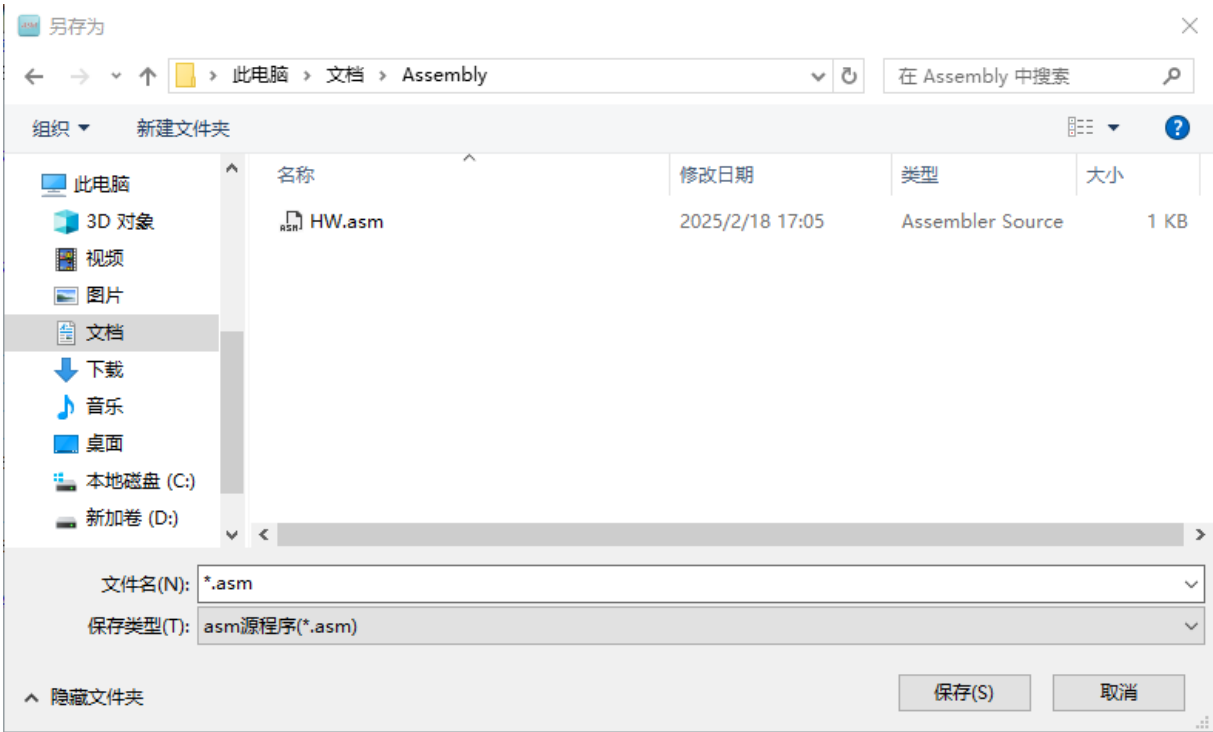
(1) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

① 打开 MASM 集成实验环境



图表 13

② 保存汇编程序



图表 14

③ 运行汇编程序



图表 15

【实验心得】

1. 汇编程序与其他高级语言的异同。

不同于其他高级语言的变量，并不直接指向某个特殊的寄存器，汇编语言直接操作寄存器，虽然好像丧失灵活性，但是这却像戴着镣铐在刀尖上跳舞，更加精确，在某些特定环境下，方便我们从底层对寄存器控制。例如，在内存资源匮乏的微机上，汇编语言的地位无疑是不可替代的。但是，汇编语言的代码并没有其他高级语言那样广泛的适用性，它对于不同的芯片，其寄存器关键字，指令都可能不一样。

2. 汇编程序三个段顺序排列的规范

对于汇编代码，我很好奇每个段的顺序对实际代码的执行会不会有什么影响，因此，我对代码进行了以下修改：

①将栈段放在 Data 段下方：


```

1  DATAS SEGMENT
2      STRING DB 13,10,'Hello World!',13,10,'$'
3  DATAS ENDS
4
5  CODES SEGMENT
6      ASSUME CS:CODES,DS:DATAS
7  START:
8      MOV AX,DATAS
9      MOV DS,AX
10     LEA DX,STRING
11     MOV AH,9
12     INT 21H
13
14     MOV AH,4CH
15     INT 21H
16 CODES ENDS
17     END START
18
19 STACKS SEGMENT STACK ;栈段
20     DW 256 DUP(?) ;注意256个字
21 STACKS ENDS
    
```

将栈段移到CODE段下方

图表 19

发现其编译无问题，而连接出现了一个警告（老朋友）（虽然也是执行下来没问题）：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
C:\>masm HW.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [HW.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:

51766 + 464778 Bytes symbol space free
0 Warning Errors
0 Severe Errors
C:\>link HW.obj

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [HW.EXE]:
List File [NUL.MAP]:
Libraries [LIB]:
LINK : warning L4021: no stack segment
C:\>_
    
```

编译无问题

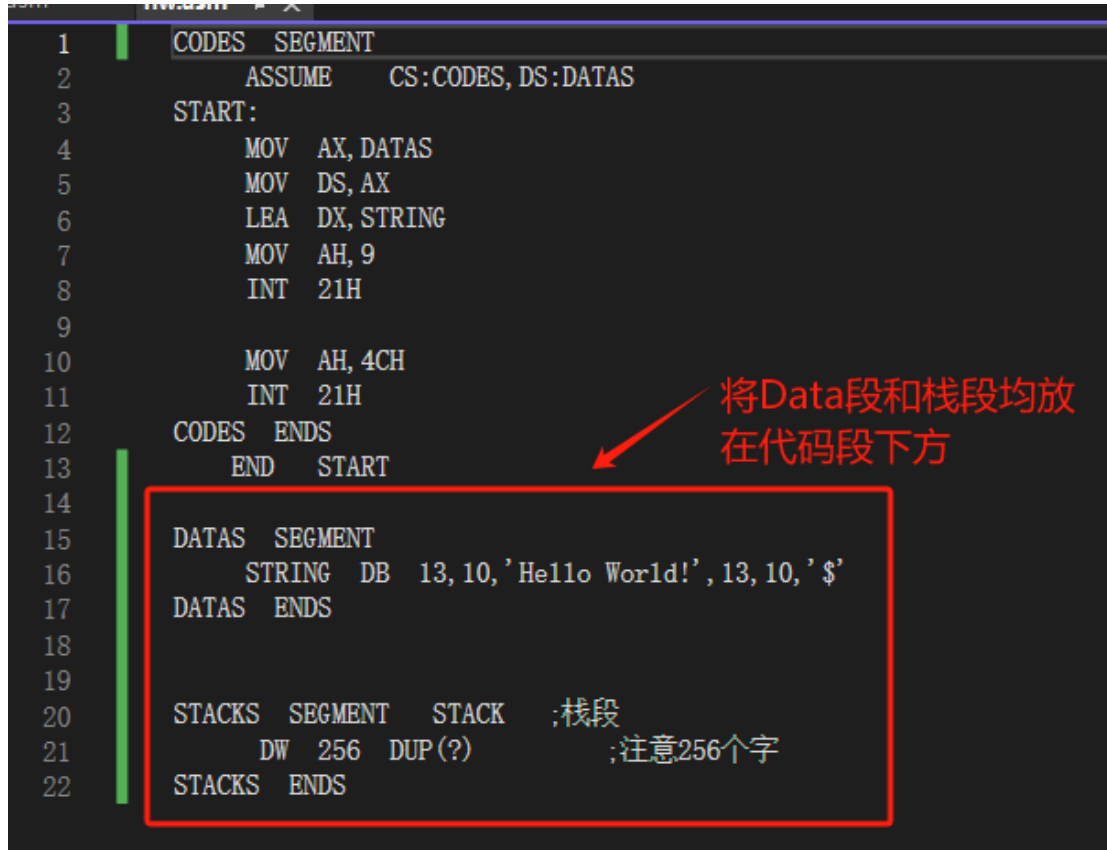
未识别栈段

图表 20

这是因为：链接器发出警告是因为它期望堆栈段在代码段之前定义。这是因为在传统的 DOS 环境下，堆栈段通常需要在程序开始执行前被正确初始化。如果堆栈段在代码段之后定义，链接器可能会认为堆栈尚未正确设置。但是 DOSBOX 环境下可能自动配置

了栈段，使其能够运行。

③将数据段和栈段放在 Code 段下方：



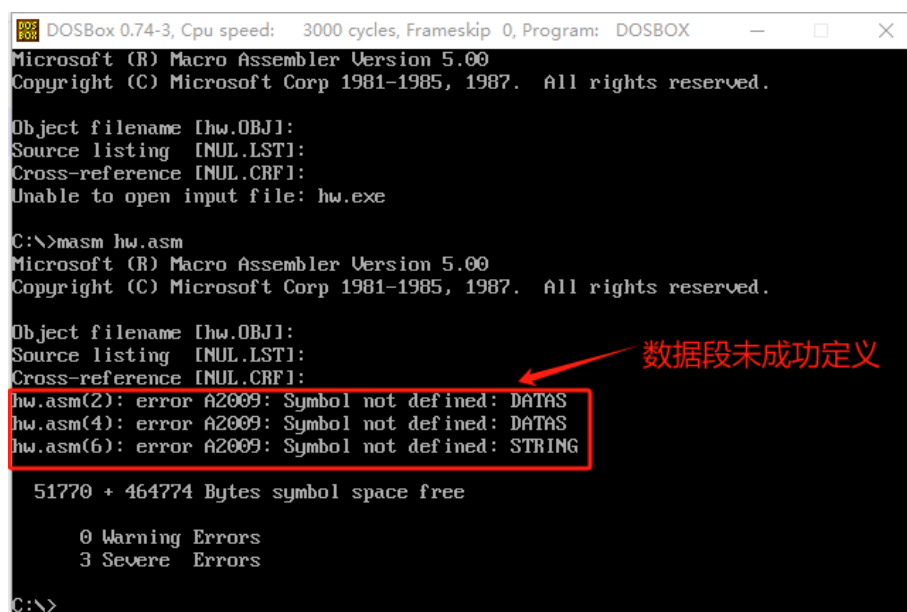
```

1  CODES SEGMENT
2      ASSUME CS:CODES, DS:DATAS
3  START:
4      MOV AX, DATAS
5      MOV DS, AX
6      LEA DX, STRING
7      MOV AH, 9
8      INT 21H
9
10     MOV AH, 4CH
11     INT 21H
12 CODES ENDS
13     END START
14
15     DATAS SEGMENT
16         STRING DB 13, 10, 'Hello World!', 13, 10, '$'
17     DATAS ENDS
18
19
20     STACKS SEGMENT STACK ;栈段
21         DW 256 DUP(?) ;注意256个字
22     STACKS ENDS
    
```

将Data段和栈段均放在代码段下方

图表 21

编译发现出现了三个错误，同时没有生成 .obj 文件



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [hw.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
Unable to open input file: hw.exe

C:\>masm hw.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

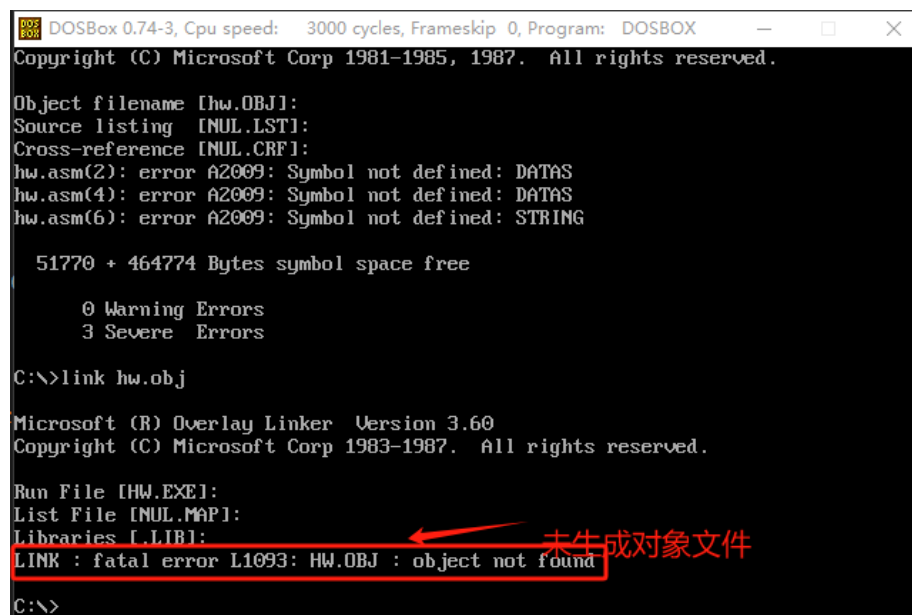
Object filename [hw.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
hw.asm(2): error A2009: Symbol not defined: DATAS
hw.asm(4): error A2009: Symbol not defined: DATAS
hw.asm(6): error A2009: Symbol not defined: STRING

51770 + 464774 Bytes symbol space free

0 Warning Errors
3 Severe Errors

C:\>
    
```

图表 22



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [hw.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
hw.asm(2): error A2009: Symbol not defined: DATAS
hw.asm(4): error A2009: Symbol not defined: DATAS
hw.asm(6): error A2009: Symbol not defined: STRING

51770 + 464774 Bytes symbol space free

0 Warning Errors
3 Severe Errors

C:\>link hw.obj

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Run File [HW.EXE]:
List File [NUL.MAP]:
Libraries [LIB1]:
LINK : fatal error L1093: HW.OBJ : object not found

C:\>
    
```

图表 23

这是由于 START 是程序的入口点，表示程序从这里开始执行。如果数据段放在 START 后面，会导致逻辑上的混乱，因为程序在执行时需要先访问数据段中的内容。当编译器发现找不到引用的变量是就会报错，也就是第 2，4，6 行中的数据。

3. 隐性声明栈段与显式声明栈段的区别

在实验中，老师给了我们两种 Hello World! 程序的写法，在我们探究过后，我们发现实际上是两种配置栈段的方法，一种是依赖于 stack 指令自动配置栈段与初始化相应的寄存器，另一种却是手动配置 stack 段，初始化 SS, SP 寄存器。

进一步了解，每个段都有其对应的寄存器，如 CODE 段—>CS, DATA 段—>DS，以

指向正确的内存单元以执行代码（这是因为在冯诺依曼结构中程序是存储在内存中的，同时 x86 是以段地址+偏移地址来作为地址的，因此其需要段地址指向程序）。

再上网查询手动配置栈段与自动配置栈段有何优劣后，我总结如下：

- 写法一：
 - 更适合需要精确控制栈段初始化的复杂程序。
 - 如果程序需要动态调整栈大小或栈位置，写法一提供了更大的灵活性。
- 写法二：
 - 更适合简单程序或快速开发场景。
 - 如果栈段的默认初始化满足需求，则写法二可以减少冗余代码。

4. Windows 集成实验环境与 DosBox 虚拟环境的代码编写感受.

Windows 集成实验环境相比于 DosBox 环境可谓是简单不少，可以发现集成实验环境直接省去了编译与运行之间的中间过程，十分简洁，而 DosBox 环境较为完整地展示了汇编程序是如何编译，连接最后转化为可执行文件的，这让我联想到 Linux 系统，它也是如此控制程序运行的，但是对于代码的调试无疑是 Windows 集成实验环境更为方便的。

5. 汇编指令总结：

1. SEGMENT 和 ENDS

- SEGMENT：用于定义一个段（segment），段是内存的一部分，通常分为堆栈段、数据段和代码段。
- ENDS：表示段定义的结束。

2. DW 和 DUP

- DW：定义字（Define Word）。表示分配一个或多个字（16 位）的存储空间。
- DUP(?)：表示重复分配指定数量的未初始化空间。例如：
 - DW 256 DUP(?) 分配了 256 个字（即 512 字节）的未初始化空间。

3. DB

DB：定义字节（Define Byte）。表示分配一个或多个字节的存储空间。

-STRING DB 13, 10, 'Hello World!', 13, 10, '\$' 定义了一个字符串变量 `STRING`，其中：

- 13 和 10 是 ASCII 码，分别表示回车（Carriage Return）和换行（Line Feed）。
- Hello World! 是要显示的文本内容。
- '\$' 是 DOS 系统中字符串的结束标志。

4. ASSUME

- ASSUME: 告诉汇编器某个寄存器指向哪个段。
- 格式: ASSUME 寄存器:段名

6. LEA

- LEA: 加载有效地址 (Load Effective Address)。用于将变量的偏移地址加载到寄存器中。
- 格式: LEA 寄存器, 变量

7. INT 和 AH

- INT: 中断调用指令, 用于调用中断服务程序。
- AH: 寄存器的一部分, 用于设置中断功能号。
- `MOV AH, 9`: 设置中断功能号为 9, 表示输出字符串。
- `INT 21H`: 调用 DOS 中断 21H, 执行功能号 9 的操作 (显示字符串)。

【课堂补充】



知识点

① CISC (Complex Instruction Set Computing) 和 RISC (Reduced Instruction Set Computing) 是计算机体系结构中的两种主要指令集架构设计方法。

CISC:

- CISC (复杂指令集计算) 是一种指令集架构,
- 其特点是支持大量的复杂指令。每条指令可以完成多个低级操作, 例如加载数据、执行运算和存储结果。指令集庞大, 包含许多复杂指令。
- 每条指令可能需要多个时钟周期来完成。
- 使用微代码 (Microcode) 来实现复杂指令。
更依赖硬件实现复杂功能。

RISC:

- RISC (精简指令集计算) 是一种指令集架构
- 指令集较小, 指令功能单一。
- 每条指令通常只需一个时钟周期。
- 更依赖编译器优化程序代码。
- 减少对微代码的需求。

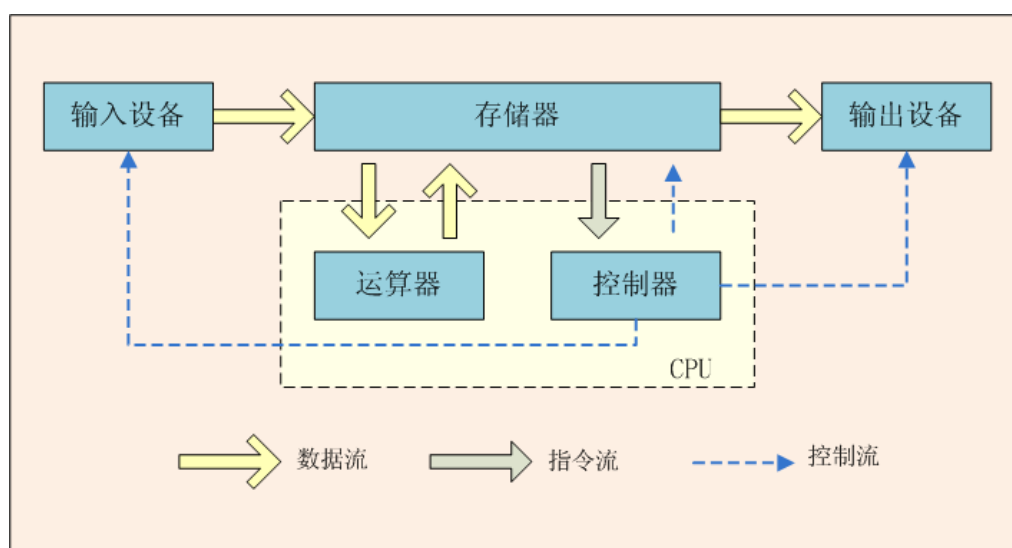
其中，我们现在所学习的 Intel8086 正是 CISC 架构下的 CPU。

②冯诺伊曼结构

是现代计算机体系结构的基础之一，由数学家约翰·冯·诺依曼（John von Neumann）在 20 世纪 40 年代提出。它是一种通用计算机的设计模型，定义了计算机的基本组成部分及其工作原理。

冯·诺依曼结构的主要特点

1. 存储程序：
 - 程序和数据都以二进制形式存储在内存中。
 - 计算机可以将程序作为数据来处理，这意味着程序也可以被修改或操作。
2. 五大组成部分：



③主板上的南桥与北桥

南桥和北桥是传统主板芯片组中的两个核心部分。北桥靠近 CPU，负责高速数据传输，主要连接 CPU、内存和显卡，管理内存访问及显卡接口等高性能需求任务；而南桥则位于远离 CPU 的位置，专注于低速 I/O 设备的通信，如 USB、SATA、音频和网络控制器等外围设备的管理。



它会在特定日期（通常是 4 月 26 日）触发，它会覆盖硬盘数据，同时尝试擦除主板 BIOS，使得计算机无法启动。这一事件暴露了当时计算机固件安全的脆弱性，从这以后，工程师推动 BIOS 设计的安全升级（如写保护和双重 BIOS）以及网络安全意识和技术的显著提升，同时也促使法律对恶意软件制作传播加强监管。